

CRYPTOSIM PRO

Web-Based Cryptocurrency Trading Platform

Lilou CONSTANTIN - Dev Data Base / Front End

Lucas NGYEN - Dev Back End

Roméo PIVAT- Dev Front End

Hugo DE SOUSA DIAS - Dev Back End

SUMMARY - CRYPTOSIM PRO

1.	Introduction	3
1.1.	Project Context and Objectives	3
1.2.	Core Features Overview.....	3
2.	System Architecture	4
2.1.	Technology Stack Selection	4
2.2.	N-Tier Backend Architecture & Data Flow	5
		6
2.3.	Real-Time Event Engine	6
3.	Database Design.....	8
3.1.	Relational Schema & Structural Mapping	8
3.2.	Data Persistence Strategy.....	9
4.	Testing Strategy	10
4.1.	REST API and WebSocket Validation	10
4.2.	Scenario-Based Testing with Bruno	11
7.	Screenshots	12
7.1.	Front-end Interface and User Views	12
7.2.	Backend Execution Logs and HTTP Responses	15
8.	Challenges Encountered	16
8.1.	Service Lifecycle Conflicts (Resolving the Singleton vs. Scoped dependency issue in the Worker)	16
8.2.	High-Frequency Filtering and Performance Optimization	16
8.3.	WebSocket Network Security	16
9.	Conclusion	17
9.1.	Project Summary and Key Achievements.....	17
9.2.	Technical Scalability and Future Enhancements	17

1. Introduction

1.1. Project Context and Objectives

In the rapidly evolving landscape of financial technology, cryptocurrency trading has emerged as a major interest in retail investors. However, high market volatility presents a steep learning curve for newcomers. To address this challenge, this project presents CryptoSim Pro, a web-based cryptocurrency trading simulation platform commissioned as a functional prototype by a fintech startup. The primary business objective is to provide a risk-free environment where new investors can engage with live market dynamics, and trade using virtual currency while monitoring their previous performance and the one of the other users

From an engineering perspective, the project serves as a comprehensive synthesis of modern software development practices. The academic and technical objectives achieved over this 4-week development cycle include:

- Designing and implementing a secure, scalable relational database schema.
- Integrating external high-frequency REST APIs to capture real-time market data.
- Architecting an asynchronous, event-driven backend capable of low-latency data broadcasting.
- Building a responsive, intuitive, and professional user interface featuring advanced data visualization.
- Enforcing industry-standard authentication and authorization mechanisms to safeguard user resources.

1.2. Core Features Overview

To deliver a high-fidelity simulation experience, the platform is structured around nine Mandatory Functional Requirements (FR1 to FR9). These requirements translate into a fully unified workflow, mapped across ten dedicated application pages—including a Landing Page, Dashboard, Market View, and Leaderboard—all sharing a cohesive, responsive layout.

The core functional capabilities of the system are categorized as follows:

- **Identity & Access Management (FR1):** Secure user registration, credential hashing, and stateful session management utilizing JSON Web Tokens (JWT) for subsequent authenticated API interactions.
- **Live Market Data Integration (FR3):** Dynamic ingestion of cryptocurrency metrics (covering at least 10 major assets) powered by the CoinGecko REST API, capturing live prices, market capitalizations, and 24-hour variations.
- **Virtual Trading Engine (FR4 & FR6):** A robust transaction execution layer supporting simulated BUY and SELL orders. The engine natively validates available cash balances and owned asset quantities while automatically enforcing a realistic 0.5% transaction fee, permanently archiving records for historical audits.
- **Portfolio Diagnostics & Analytics (FR2 & FR5):** A comprehensive dashboard computing consolidated financial indicators, including Net Liquidation Value (NLV), available cash, aggregate asset allocation, and real-time unrealized profit/loss (P&L) margins.
- **Social & Engagement Metrics (FR7):** A competitive, real-time leaderboard ranking the Top 10 users globally based on their overall investment returns.
- **Advanced Tracking Features (FR8 & FR9):** A personalized **Watchlist** allowing users to follow specific assets via clean RESTful routes (`/api/WatchList/add/{cryptotld}`), coupled with a cutting-edge **Price Alert Engine**. The alert engine allows users to establish custom price thresholds (ABOVE or BELOW) and leverages WebSockets to push instant notifications the moment a target threshold is breached by the market.

2. System Architecture

2.1. Technology Stack Selection

We decided to build the project based on our previous experience and affinity with technology. As a consequence, the back end is an ASP.NET Core Web API written in C#, due to its static typing, first-class dependency-injection container, and the maturity of its data and security ecosystem which has properties that matter when the system manipulates money-like quantities and manages authentication. Persistence relies on PostgreSQL accessed through Entity Framework Core via the Npgsql provider. The front end is a single-page application built with React and Vite, using React Router for client-side navigation and Recharts for portfolio visualization. Market data is sourced from the CoinGecko public REST API.

Layer	Technology	Role & justification
Back end	ASP.NET Core Web API (C#)	Strongly-typed, high-performance API host with built-in DI and middleware pipeline.
ORM	Entity Framework Core + Npgsql	Code-first mapping between C# entities and PostgreSQL; migrations manage schema evolution.
Database	PostgreSQL	Robust relational engine; decimal(18,8) columns guarantee exact financial precision.
Security	ASP.NET Identity + JWT Bearer	Battle-tested user store with password hashing; stateless JWT auth for the SPA.
Front end	React + Vite + React Router	Component-based SPA with fast tooling and protected client-side routes.
Charts	Recharts	Declarative charts for allocation, P&L and session sparklines.
Market data	CoinGecko REST API	Free source of live prices, market cap, volume and 24h change for major assets.
API testing	Bruno	Version-controllable, scenario-based collection for the REST endpoints.

On the client side, the React application is organized for clarity and reuse. Global state is provided through three React Contexts – an AuthContext (session and JWT), a MarketDataContext (the cached price feed) and a PortfolioContext (positions and derived metrics). While a ProtectedRoute component guards authenticated pages and redirects unauthenticated visitors to the login screen. All network access flows through a single apiClient module that attaches the Bearer token to every request and normalizes the several error shapes returned by the back end (plain string arrays from Identity, { message } objects, { success, message } envelopes) into one readable error, so the UI can present failures consistently.

2.2. N-Tier Backend Architecture & Data Flow

The back end follows a strict **N-tier layered architecture (or onion architecture)** in which each responsibility lives in its own folder and communicates with adjacent layers through interfaces rather than concrete types. This separation keeps the codebase testable and lets any single layer evolve independently, which in our case was pretty useful during the development, allowing us to split tasks and step between developers and test them separately:

- **APP** — the presentation layer: API controllers (AuthController, MarketController, TradingController...) and the data-transfer objects (DTOs) that shape requests and responses ensuring the security of our backend.
- **LOGIC** — the business layer: (Ex: AuthService, CryptoPriceService, MiddleOffice BackOffice ...), each hidden behind an interface.
- **PERSIST** — the persistence layer: the AppDb Entity Framework context, the single gateway between C# objects and PostGreSQL.
- **MODEL** — the domain entities (AppUser, Portfolio, Transaction, Watchlist, PriceAlert).
- **COMMON** — cross-cutting concerns: the TradingRulesSettings options object, the ServiceException type and the ErrorCode enumeration.

All services are registered with a **scoped** lifetime in Program.cs, and trading parameters are injected through the strongly-typed IOptions<TradingRulesSettings> pattern so that configuration is read once and shared safely.

The corresponding wiring in Program.cs is deliberately concise — each interface is bound to its implementation with a scoped lifetime, and the trading rules are bound from configuration:

```
// Program.cs — dependency-injection container
builder.Services.AddDbContext<AppDb>(o =>
    o.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddIdentity<AppUser, IdentityRole>()
    .AddEntityFrameworkStores<AppDb>()
    .AddDefaultTokenProviders();

builder.Services.Configure<TradingRulesSettings>(
    builder.Configuration.GetSection("TradingRules"));

builder.Services.AddScoped<IMiddleOffice, MiddleOffice>();
builder.Services.AddScoped<IBackOffice, BackOffice>();
builder.Services.AddScoped<IAuthService, AuthService>();

builder.Services.AddScoped<IAuthService, AuthService>();
```

Example of the dataflow of a transaction:

1. The React client sends a **POST** to `/api/Trading/execute` with a `TradeRequestDto` (`CryptoId`, `Type`, `Quantity`) and a **Bearer JWT** in the Authorization header.
2. **TradingController** extracts the user identity from the token's **NameIdentifier** claim and loads the user together with their portfolio positions.
3. **CryptoPriceService** returns the current market snapshot, served either from a 30-second in-memory cache or, on a cache miss, from CoinGecko.
4. **MiddleOffice.ValidateTrade** runs the pre-trade checks: sufficient cash for a BUY (fees included), the global exposure limit, and — for a SELL — position existence and a sufficient owned quantity, which blocks short-selling.
5. On approval, the execution price is frozen and **BackOffice.ExecuteTradeAsync** settles the order: it debits or credits cash, recalculates the weighted-average buy price, updates or removes the portfolio line, and appends an immutable **Transaction** record.
6. Every database mutation is committed atomically through a single **SaveChangesAsync** call, and a structured `{ Success, Message }` result is returned to the client.

Authentication is **stateless**: on login the **AuthService** verifies the credentials through ASP.NET Identity and issues a signed JWT carrying the user's identity claims. The token is signed with HMAC-SHA256 and expires after two hours; the client stores it and replays it on every subsequent request, where the `[Authorize]` pipeline validates the signature before the controller runs.

```
// AuthService.LoginUser – JWT issuance
var claims = new[] {
    new Claim(JwtRegisteredClaimNames.Sub, user.Id),
    new Claim(JwtRegisteredClaimNames.Email, user.Email!),
    new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
};
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret));
var descriptor = new SecurityTokenDescriptor {
    Subject = new ClaimsIdentity(claims),
    Expires = DateTime.UtcNow.AddHours(2),
    SigningCredentials = new SigningCredentials(
        key, SecurityAlgorithms.HmacSha256Signature)
};
```

2.3. Real-Time Event Engine

Real-time behaviour rests on two mechanisms.

→ First, market data is **pulled** from CoinGecko and **cached** in a shared **IMemoryCache** entry with a 30-second time-to-live. Because the cache is shared across all users rather than per-request, dozens of concurrent reads collapse into at most one external call per interval, which allow us to bypass the critical problem of the fact that we are limited to a certain amount of request per minute (and also month on the app). We have decided to track ten major assets: Bitcoin, Ethereum, Solana, Ripple, Cardano, Dogecoin, Polkadot, Litecoin, Chainlink and Stellar.

```
// CryptoPriceService – the cache short-circuits the network
if (_cache.TryGetValue(MarketPricesCacheKey,
    out IEnumerable<CryptoPriceDto> cached))
    return cached; // instant, no HTTP call

var response = await _httpClient.GetAsync(baseUrl);
response.EnsureSuccessStatusCode();
var data = await response.Content
    .ReadFromJsonAsync<Dictionary<string, CoinGeckoSimplePriceData>>();
// ... project each entry into a CryptoPriceDto ...
_cache.Set(MarketPricesCacheKey, result, TimeSpan.FromSeconds(30));
```

Second, a **background worker** (implemented as a hosted BackgroundService) periodically compares live prices against every user's active PriceAlert thresholds. When a threshold is breached in the configured direction (ABOVE or BELOW), the alert is marked inactive and a notification is delivered to the owning client. The delivery transport is designed around **WebSockets** so that the browser receives events without polling thanks to SignalR.

3. Database Design

3.1. Relational Schema & Structural Mapping

The schema builds on the tables provided by **ASP.NET Identity** (the `AspNetUsers` / `AspNetRoles` family). The application user, `AppUser`, **extends** `IdentityUser` with a first name, surname, an initial virtual `CashBalance` of 100,000 units, and a creation timestamp. Four domain tables hang off the user through a one-to-many relationship, each linked by a **`UserId` foreign key**:

Entity	Key fields	Purpose
AppUser	Id, Name, Surname, CashBalance, CreatedAt	Extended Identity user holding the virtual cash balance.
Portfolio	Id, UserId, Cryptoid, Quantity, AvgBuyPrice	Open positions with weighted-average cost basis (PRU).
Transaction	Id, UserId, Cryptoid, Type, Quantity, Price, Fee, CreatedAt	Immutable, append-only ledger of every executed order.
Watchlist	Id, UserId, Cryptoid	Assets a user has chosen to follow.
PriceAlert	Id, UserId, Cryptoid, TargetPrice, Direction, IsActive	User-defined price thresholds evaluated by the worker.

Concretely, the extended user and a single portfolio line are declared as plain C# classes; Entity Framework turns them into tables:

Example for the AppUser class/table:

```
public class AppUser : IdentityUser
{
    public string Name { get; set; } = "";
    public string Surname { get; set; } = "";
    public decimal CashBalance { get; set; } = 100000m;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public ICollection<Portfolio>? Portfolios { get; set; }
    public ICollection<Transaction>? Transactions { get; set; }
}

public class Portfolio
{
    [Key] public int Id { get; set; }
    public required string UserId { get; set; }
    public required string CryptoId { get; set; }
    public decimal Quantity { get; set; }
    public decimal AvgBuyPrice { get; set; } // weighted-average cost (PRU)
}
```

All monetary and quantity columns are mapped to `decimal(18,8)` at the model-builder level. Eight fractional digits are necessary both to represent fractional crypto quantities faithfully and to avoid the rounding drift inherent to floating-point types — a non-negotiable requirement for any system that behaves like a ledger.

```
protected override void OnModelCreating(ModelBuilder b)
{
    base.OnModelCreating(b);
    b.Entity<AppUser>().Property(u =>
u.CashBalance).HasColumnType("decimal(18,8)");
    b.Entity<Portfolio>().Property(p => p.Quantity)
.HasColumnType("decimal(18,8)");
    b.Entity<Portfolio>().Property(p =>
p.AvgBuyPrice).HasColumnType("decimal(18,8)");
    b.Entity<Transaction>().Property(t => t.Price)
.HasColumnType("decimal(18,8)");
    b.Entity<Transaction>().Property(t => t.Fee)
.HasColumnType("decimal(18,8)");
}
```

3.2. Data Persistence Strategy

Persistence follows the **Code-First** philosophy:

- the C# model is the single source of truth, and the database schema evolves through Entity Framework **migrations** (InitialIdentitySetup → AddTradingModels → ...). The AppDb context also acts as the application's **unit of work**: changes tracked in memory are only written when a single SaveChangesAsync succeeds, giving **all-or-nothing settlement** — if any part of an order fails, no partial state reaches the database.

Two further guarantees are worth highlighting. The Transaction table is strictly **append-only**, forming an auditable history that is never mutated after the fact. And on every additional BUY, AvgBuyPrice is recomputed as a **cost-basis weighted average** of the existing and incoming lots, so realized and unrealized P&L remain accurate as a position is built up over time. To protect balances and positions from **race conditions** when two orders for the same user arrive nearly simultaneously, an **optimistic-concurrency** strategy was introduced so that conflicting writes are detected and rejected rather than silently overwriting one another.

The cost-basis recalculation and the atomic settlement it belongs to are captured in a few lines of the Back Office:

```
// BackOffice – settling an additional BUY into an existing line
decimal totalQuantity = existing.Quantity + request.Quantity;
decimal totalCostBasis = existing.Quantity * existing.AvgBuyPrice
                        + request.Quantity * executionPrice;

existing.AvgBuyPrice = totalCostBasis / totalQuantity; // new weighted PRU
existing.Quantity    = totalQuantity;

_context.Transactions.Add(new Transaction { /* immutable ledger entry */ });
await _context.SaveChangesAsync();           // atomic settlement
```

4. Testing Strategy

4.1. REST API and WebSocket Validation

Validation is enforced in **two complementary layers**.

- The first is **transport / DTO validation** driven by on TradeRequestDto: the crypto identifier and order type are required, the type must match the regular expression (BUY|SELL), and the quantity must be strictly positive. Violations are rejected automatically with an HTTP 400 before any business logic runs.
- The second is **domain validation** inside the MiddleOffice, which owns the financial rules (cash sufficiency, exposure limits and the short-selling ban). Every trading endpoint is protected with the [Authorize] attribute, so requests without a valid JWT are refused with a 401.

The DTO contract makes the first layer declarative: the framework rejects a malformed order before it ever reaches a service:

```
public class TradeRequestDto
{
    [Required] public string CryptoId { get; set; } = "";

    [Required]
    [RegularExpression("(BUY|SELL)")]
    public string Type { get; set; } = "";

    [Required]
    [Range(0.00000001, double.MaxValue)]
    public decimal Quantity { get; set; }
}
```

The second layer, in the Middle Office, is where the financial guardrails live: cash sufficiency, the global exposure limit, and the short-selling ban:

```
public class TradeRequestDto
{
    [Required] public string CryptoId { get; set; } = "";

    [Required]
    [RegularExpression("^(BUY|SELL)$")]
    public string Type { get; set; } = "";

    [Required]
    [Range(0.00000001, double.MaxValue)]
    public decimal Quantity { get; set; }
}
```

4.2. Scenario-Based Testing with Bruno

The Bruno collection is organized into authentication, market-data and trade-execution folders. Crucially, it exercises not only the **happy paths** but also each **guardrail**, confirming that invalid orders are rejected with a clear message. The main scenarios are summarized below.

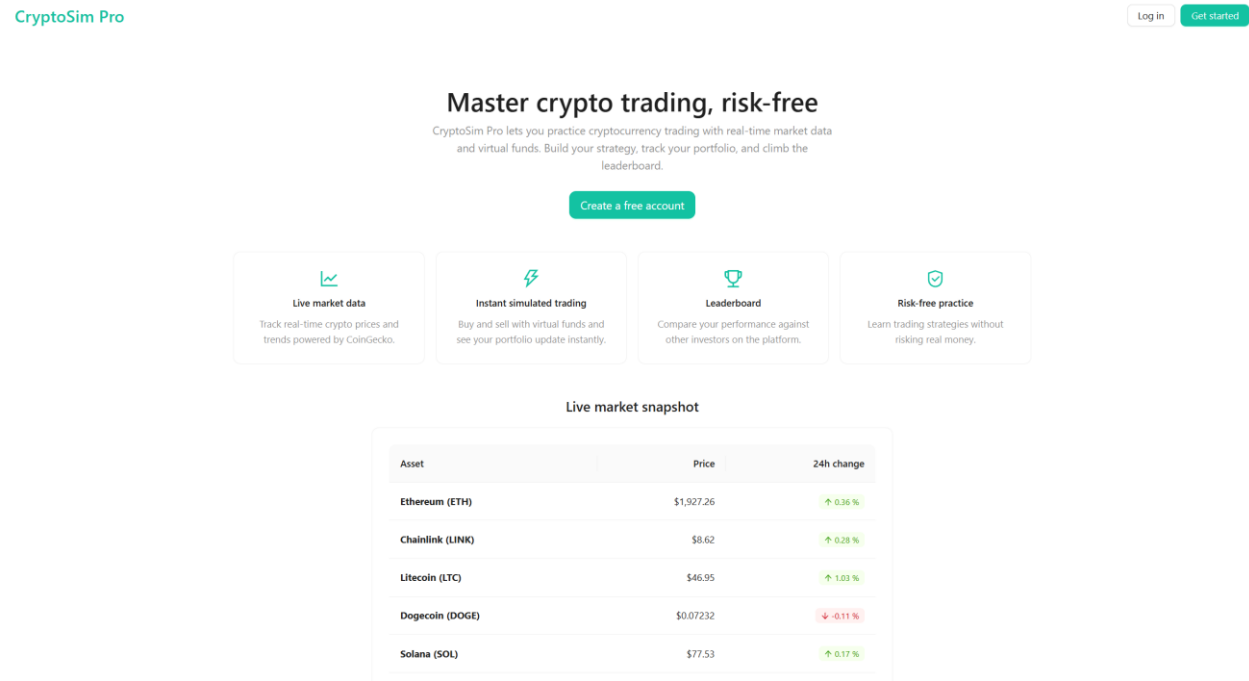
Scenario	Request	Expected result
Register / Login	POST /api/Auth/register, /login	201/200 and a signed JWT returned.
Market snapshot	GET /api/Market/marketprices	200 with the ten tracked assets.
Valid buy	BUY 0.5 bitcoin	200 — order settled, cash debited, position opened.
Valid sell	SELL an owned quantity	200 — order settled, cash credited net of fee.
Insufficient funds	BUY 50 bitcoin	400 — cash-available check fails.
Short-selling blocked	SELL 10 ethereum (not owned)	400 — position does not exist.
DTO validation	malformed / negative-quantity body	400 — DataAnnotations reject the payload.
Transaction history	GET /api/Trading/history	200 — ledger ordered most-recent-first.

7. Screenshots

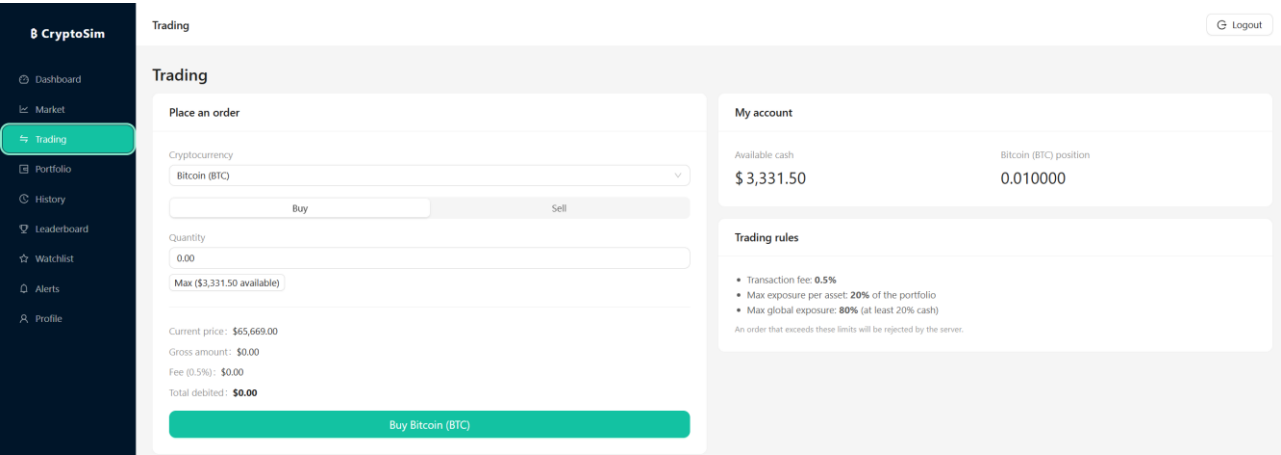
7.1. Front-end Interface and User Views

The following figures illustrate the ten application pages and the cohesive, responsive layout shared across the SPA. Final captures will be inserted in place of the placeholders below.

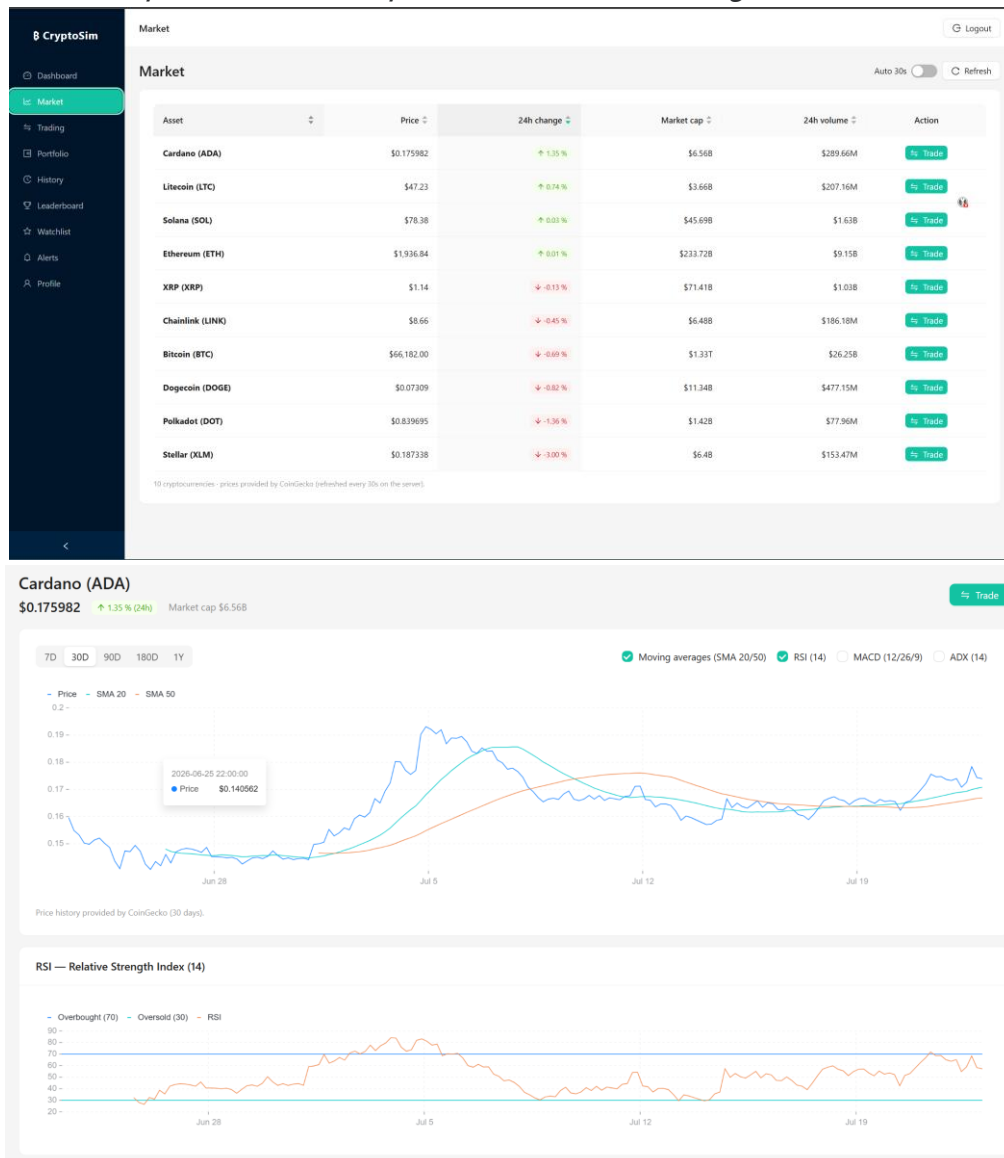
Landing page – product value proposition and call-to-action.



Trading – Net Liquidation Value, available cash, allocation donut and unrealized P&L.



Market view – live prices, market cap, 24h volume and change for the ten tracked assets.



Trading— order ticket (BUY / SELL, quantity) with real-time price and fee preview.

Trading

Place an order

Cryptocurrency: Bitcoin (BTC)

Buy Sell

Quantity: 0.00

Max (\$3,331.50 available)

Current price: \$65,671.00

Gross amount: \$0.00

Fee (0.5%): \$0.00

Total debited: \$0.00

Buy Bitcoin (BTC)

My account

Available cash: \$3,331.50

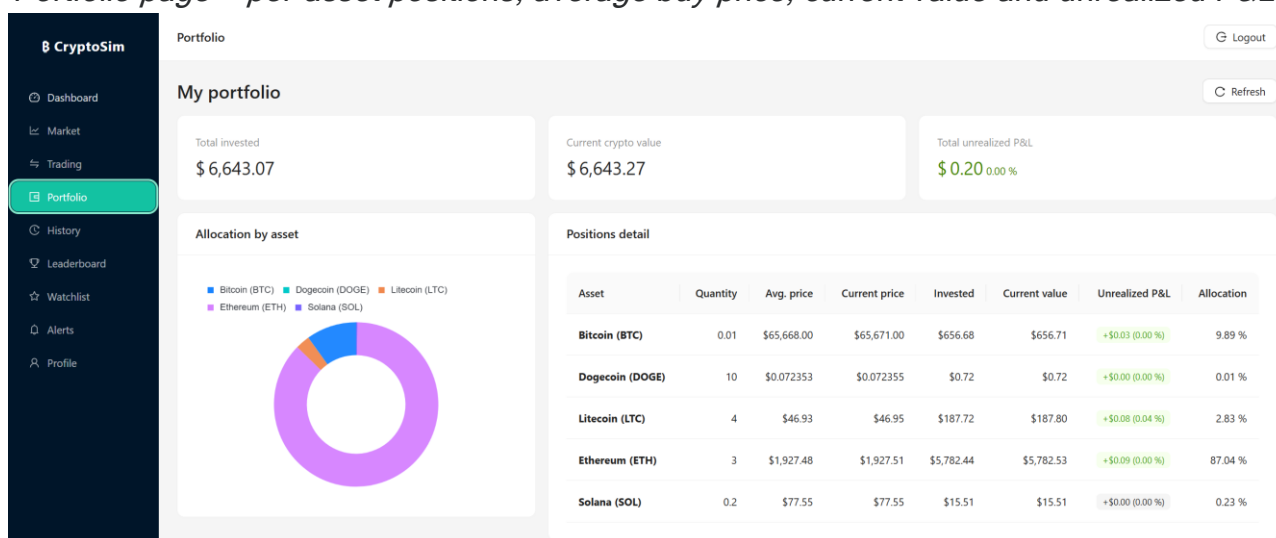
Bitcoin (BTC) position: 0.010000

Trading rules

- Transaction fee: 0.5%
- Max exposure per asset: 20% of the portfolio
- Max global exposure: 80% (at least 20% cash)

An order that exceeds these limits will be rejected by the server.

Portfolio page – per-asset positions, average buy price, current value and unrealized P&L.



Leaderboard – global ranking of the top users by overall return. forms with client-side validation.

Rank	User	Total value	Crypto value	Total P&L	Return	Activity
1	Hugo DE SOUSA DIAS	\$101,244.12	\$54,464.31	+\$1,636.97	3.10 %	6 orders
2	Ilona DE SOUSA DIAS	\$9,974.78	\$6,643.27	+\$0.20	0.00 %	6 orders
3	Hugo DESOUASDIAS	\$0.00	\$0.00	+\$0.00	0.00 %	0 order

Authentication – registration and login forms with client-side validation.

7.2. Backend Execution Logs and HTTP Responses

On the server side, structured console logging traces cache hits, external calls and the outcome of each order, while the API returns consistent JSON envelopes. A representative successful execution response is shown below.

```
POST /api/Trading/execute      200 OK
{
  "Success": true,
  "Message": "L'ordre de BUY pour 0.5 bitcoin a été exécuté avec succès."
}

[Cache] Market prices served from memory (age < 30s)
```

8. Challenges Encountered

8.1. Service Lifecycle Conflicts (Resolving the Singleton vs. Scoped dependency issue in the Worker)

The alert worker is a hosted **BackgroundService**, which the runtime instantiates **once**, as a singleton, for the whole lifetime of the application. The Entity Framework context (AppDb), by contrast, is registered with a scoped lifetime because it is not thread-safe and is designed to be short-lived. Injecting the scoped AppDb directly into the singleton worker triggers the classic startup error *“cannot consume scoped service AppDb from singleton”*.

The resolution is to inject **IServiceScopeFactory** into the worker and, on each evaluation cycle, create a fresh scope — using `var scope = _scopeFactory.CreateScope();` from which a correctly-scoped AppDb is resolved.

This gives the long-lived worker per-iteration database access without leaking lifetimes or sharing a context across threads.

```
// Resolving a scoped DbContext inside a singleton BackgroundService
using (var scope = _scopeFactory.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDb>();
    var alerts = await db.PriceAlerts
        .Where(a => a.IsActive).ToListAsync();
    // ... compare against live prices, deactivate & notify on breach ...
}
```

8.2. High-Frequency Filtering and Performance Optimization

CoinGecko's demo tier imposes a tight call budget (on the order of a few dozen calls per minute). With several users polling the market and every trade requiring a fresh price, naïve per-request calls would exhaust that budget within seconds and cause failures. The solution is the single shared **IMemoryCache** entry with a 30-second TTL: all users read from the same cached snapshot, collapsing many would-be external calls into at most one per interval.

Within the hot path of a trade, the price list is projected into a **dictionary keyed by asset id** for $O(1)$ lookups, and the whole portfolio is valued in a single in-memory pass. Together these choices keep price resolution off the network and off the database wherever possible, which is what makes validation feel instantaneous.

8.3. WebSocket Network Security

An always-open push channel widens the attack surface, so the alert socket is designed around three principles.

- First, the connection is **authenticated with the same JWT** used by the REST API rather than left anonymous.
- Second, allowed origins are **restricted through the existing CORS policy**, so only the trusted SPA origin may open a socket.
- Third, every message is **scoped to its owner**, so a client can only ever receive its own alert notifications and never another user's.

9. Conclusion

9.1. Project Summary and Key Achievements

Within a four-week cycle, the project delivered a working prototype covering the mandatory requirements:

- secure Identity-plus-JWT authentication;
- live ingestion of a ten-asset market with shared caching;
- a validated virtual trading engine that applies a realistic fee and enforces cash,
- exposure and short-selling rules;
- portfolio analytics spanning NLV,
- cash, allocation and unrealized P&L;
- a global leaderboard;
- and watchlist and price-alert features.

All of this sits behind a clean N-tier back end that mirrors a real trade lifecycle: Front, Middle and Back Office and a responsive React front end.

The most valuable outcome is not any single feature, but the **financial correctness and separation of concerns** baked into the design: exact decimal arithmetic, an immutable ledger, atomic settlement, and risk validation kept independent from booking.

9.2. Technical Scalability and Future Enhancements

Several avenues would take the prototype toward production readiness:

- Replace REST polling with streaming exchange WebSocket feeds for genuinely real-time prices.
- Move alert evaluation to a horizontally-scalable worker fed by a message queue, and adopt a distributed cache (Redis) so both caching and alerts survive across multiple instances.
- Add richer order types (limit, stop-loss) and partial-fill semantics.
- Introduce refresh-token rotation with shorter access-token lifetimes, and enable JWT issuer/audience validation for production.
- Paginate the history and market endpoints and add server-side leaderboard aggregation.
- Containerize the stack with Docker, add a CI/CD pipeline, and instrument the system with structured logging, metrics and tracing.
- Complement the Bruno collection with automated integration tests (e.g. xUnit) run on every commit.